What's new

Old and new apps, via modern coding agents

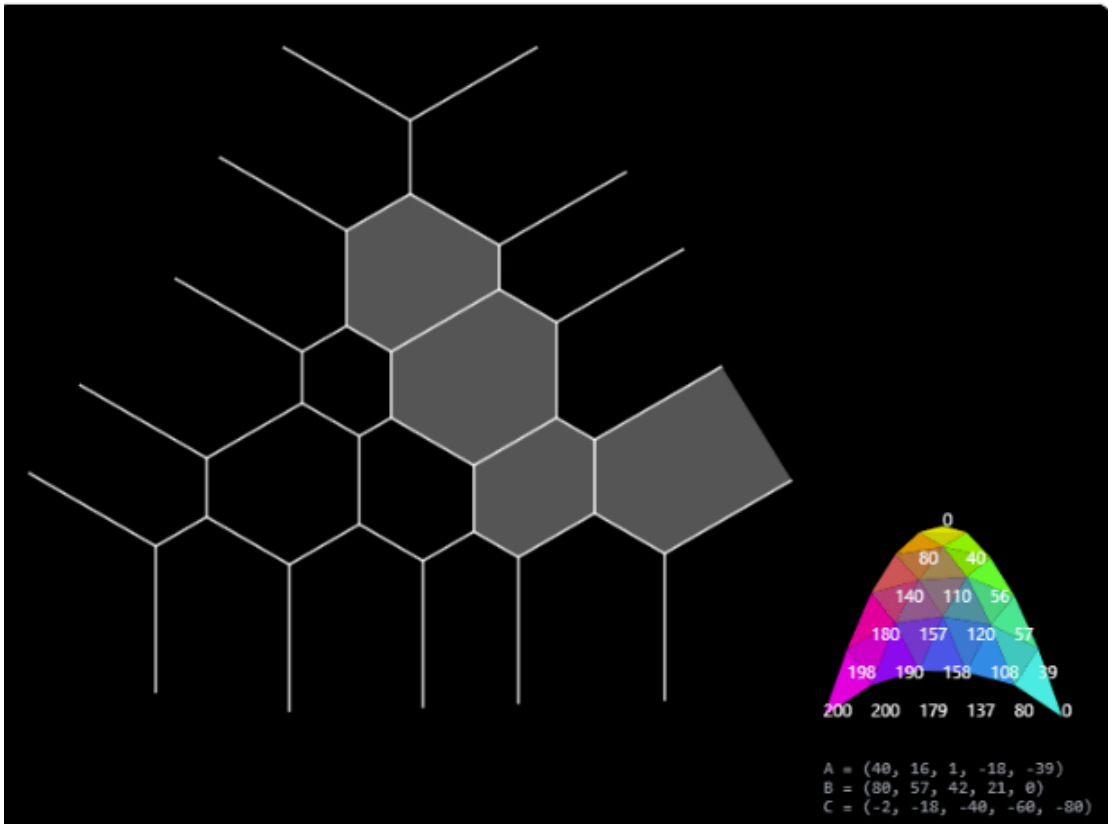
Terence Tao · 11 July 2026

https://terrytao.wordpress.com/2026/07/11/old-and-new-apps-via-modern-coding-agents/

tao2typ · converted to Typst 2026-10-14 (UTC)

I have been interested in machine-assisted ways to do and teach mathematics from as far back as 1999, when I started coding several applets in Java 1.0, both for my complex analysis and linear algebra courses, as well as to visualize various mathematical objects I was interested in (such as honeycombs or Besicovitch sets). This was moderately successful; but the applets were time-consuming to program. Eventually, the standards for web pages stopped supporting this version of Java, and the applets became non-functional.

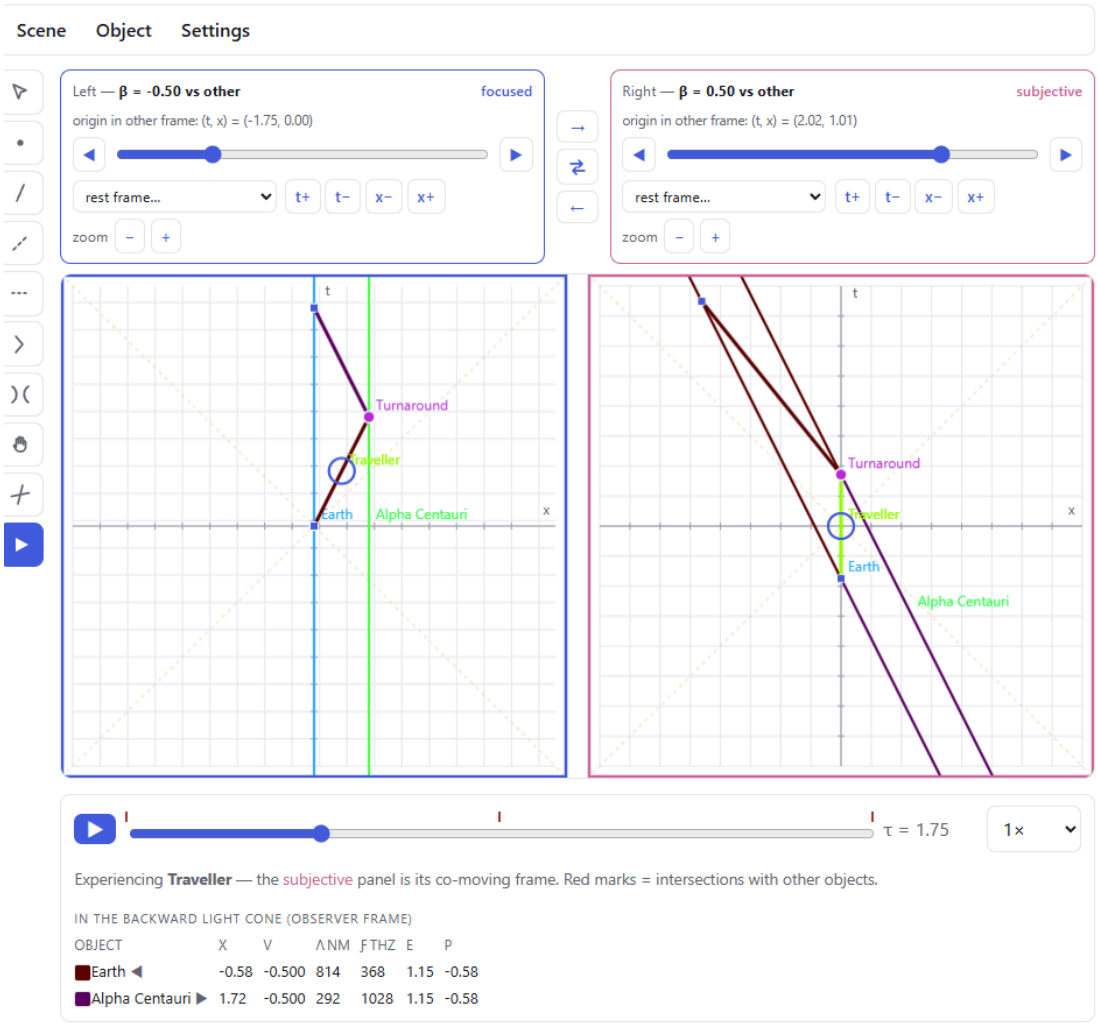
However, in the last few days I have begun the process of migrating much of my old web page and blog data to a more maintainable repository, using modern AI assistance. As an experiment, I asked the agent to port my old applets to a modern supported language (we landed on Javascript), and it managed to do so in a matter of hours, with all of my old applets now functional again, with even a few graphical upgrades (for instance, the Besicovitch set applet is now colorized, in contrast to my original monochrome version). I am particularly pleased to see the honeycomb applet that I wrote with Allen Knutson in 1999 come back to life, as this was a particularly tricky one to code by hand:



Notoriously, LLM-based coding agents can create various blatant or subtle bugs in their code; but in the porting of these two dozen or so applets, I could only find one minor bug (the handling

of a drag event in one of the complex analysis applets had unwanted behavior when dragging outside of the main box), and in fact the agent identified two bugs in the original code that I was not aware of, so it ended up being a net wash as far as code quality was concerned. In any event, as these applets are meant to be secondary visual aids rather than critical components of a mathematical argument, the downside risk of such bugs is relatively low.

The process was painless enough that I decided to also try coding some new apps, in addition to porting the old ones. Back in 1999 I had an ambitious idea for a visualization tool for special relativity; this was before the release of the software tool Inkscape, but the idea I had in mind was basically "Inkscape, but in Minkowski space". I had even started writing Java code for this app, but the code complexity became too much for me, and I abandoned the project. However, after a couple hours of "vibe coding" with an AI agent, I was finally able to generate an applet that matched the vision I had back in 1999, which can now be found here. A summary of the conversation I had with the agent to generate this code can be found here (it has been edited down to remove a large number of tedious technical implementation reports). While I have playtested the app somewhat, I would be interested in receiving further feedback on this "alpha" version of the applet, as I am sure (especially given the LLM-generated nature of the code) that there are still some bugs and rough edges to be ironed out.



After writing my blog post on the Gilbreath conjecture paper earlier today, I realized that I could similarly ask the agent to code a visualization tool for the Gilbreath conjecture to accompany

the paper and blog post. After another few hours of conversation, this is now done; you can try out the visualization here. Again, the procedure was quite painless (see this transcript of the process), and I think I may add such interactive visualizations as supplements for future papers; as such supplements are not mission-critical to the core of the paper, I again feel that the downside risk of using guided interaction with LLM agents to generate such visualizations is acceptable.



Comments (11)

 ??? · 12 July 2026

I noticed a small inconsistency in the per-panel boost controls. The displayed β value and the slider appear to be relative to the other panel, via `relBeta(i)` and the slider handler.

 **Terence Tao** · 12 July 2026

Yes, this is intentional; each of the reference frames is only described in terms of the other. There is a "laboratory" rest frame in the back end of the applet which is fixed, and is used for data storage purposes to record the state of the objects and reference frames that are visible, but I have deliberately kept this frame as an implementation detail hidden from the public interface in order to keep with the spirit of the principle of relativity. (With more effort, I could also have coded the data storage system to not rely on any laboratory rest frame at all, or to add such a frame to the data model, but I'm happy with the compromise of using preferred reference frames in the backend but not in the frontend.)

 **Anonymous** · 12 July 2026

These are so beautiful! Maybe you could also add to or improve that visualization you made earlier: https://terrytao.wordpress.com/2016/05/13/visualising-random-variables/. And, since there's already an app for special relativity, would it be possible to make one for general relativity?

 **Terence Tao** · 12 July 2026

In fact I was midway through developing precisely that random variable app. Given the interest, I have published an in-progress version at https://teorth.github.io/tao-web/apps/random-variables.html , but I am still testing it, tweaking the design, and adding more features, so this should be viewed as a pre-alpha version. Any feedback would be very welcome!

 **Anonymous** · 12 July 2026

Yay!

 **Anonymous** · 12 July 2026

You mentioned "[adding] such interactive visualizations as supplements for future papers". Guessing you would provide links in papers and blog posts and host the apps on your site. How avoid the creeping obsolescence of your earlier, hand-coded applets?

Some projects, like OpenAI's Symphony, are being shipped as specification files, written in Markdown. Feeding the spec to a coding agent generates the working app.

 **Terence Tao** · 12 July 2026

When I ported my legacy Java 1.0 apps, I found that the agent was already able to understand the structure of the code just from inspecting it, as well as the HTML page containing it, and could use this to do a faithful port; my only guidance was, for the more complex apps, to build the app in stages, for instance building the math engine and test it first before buiding the GUI, but even this the agent largely handled on its own.

> Probably the fact that the apps were performing standard mathematical tasks (such as Gaussian elimination) for which the LLM already had plenty of context about from training helped with this; I also like to think that my hand-written code was already structured in a logical enough fashion that the spec could be extracted from it without too much difficulty.
>
> In general, I think that the precise language or spec that a given piece of software is written in has become far less relevant now that the friction and cost of porting from one language to another has gone down to nearly zero. As long as there is enough context provided (either through a formal spec, or other companion material, such as web pages discussing the software), and that the task being performed uses standard concepts that are already familiar to the agent, then LLMs do a perfectly adequate job of conversion.

 **Anonymous** · 12 July 2026

ok sir what's ur conclusion, is not worth to learn to code and does having domain knowledge or being software engineer has an advantage

>  **Terence Tao** · 12 July 2026
>
> Definitely I consider my programming experience (whether it has been in Java, Python, Lean, or otherwise) as well as domain expertise (in, say special relativity) essential to being able to get useful code from the agent, and to be able to detect and diagnose issues. With the relativity app, for instance, being able to distinguish the data model (in which objects are recorded with respect to a fixed laboratory frame) and the user interface (where objects are only rendered in the two current reference frames) was essential in order to get the agent to understand my intention and to capture the spirit of the principle of relativity, which was one of the key initial motivations of that project. The high level code design decisions still remain in the "vibe coding" model; it is the lower level syntax and implementation issues that have been largely automated away. But most likely any new programmer will need to spend time working those lower levels by hand to acquire a solid grasp of the higher level code abstractions.

Terence Tao AI Coding Agents Experiment: Two Dozen 1999 Java Applets Revived in Hours — and 2 Bugs He Never Knew About - Sean Kim — Arts and Tech · 13 July 2026

[…] dozen applets ported from Java to JavaScript "in a matter of hours," as he puts it in the original post, now running in any modern browser and hosted publicly on GitHub Pages. One detail worth flagging: […]

 **Jimmy** · 18 September 2026

The detail about the agent resurrecting two bugs the original applets never surfaced is the best argument in the piece for keeping the old code around: legacy artifacts double as test oracles. It also matches what heavy users of coding agents keep reporting – the interesting failures are rarely about generating code, they are about state, verification and quotas. That last one is underrated: usage windows and resets now interrupt long agentic sessions the way build times used to, which is why I track Codex reset windows on a small independent reference site (https://codexreset.today/).